SOLUTION DESIGN DOCUMENT

Mortgage Pricing Exception Management Service

Blue Harvest | Agentic SDLC Engineering Assignment

Original proposal: 2 November 2026. Current-code method appendix updated 7 November 2026.

Design basis: Blue Harvest assignment (3 pages) and Pricing Exceptions — Entity and Application Design (5 pages).

Executive summary

Design a small internal HTTP service that captures mortgage rate-discount exceptions, gives independent reviewers a controlled approval process, exposes only the currently approved discount to the mortgage application process, and keeps an auditable history. The suggested implementation is a single ASP.NET Core Web API with logical Presentation, Business, and DataAccess layers, EF Core, and SQLite for local execution.

Source distinction: “Required” denotes assignment-derived needs. “Proposed” denotes implementation decisions from the supplied entity design or recommendations introduced in this document. Business policies marked “Confirm” are not specified by the assignment.

1. Problem statement and scope

1.1 Business problem

A financial-services client originates mortgages through relationship managers. A manager may need an interest-rate discount below the standard offer. Before the application process can use that discount, an authorized reviewer must approve the specific terms. Without a controlled exception service, requests and approvals may be hard to trace, repeated upstream calls may cause duplicates, and downstream consumers may use a discount that is no longer approved.

1.2 Project objective

Provide a locally runnable, testable internal API for the end-to-end workflow: create a pricing exception for an existing mortgage application → review the submitted terms → retrieve current status/history → retrieve the current approved discount for downstream mortgage processing.

1.3 Boundaries and scope decisions

In scope (MVP)	Out of scope / integration boundary
Create, retrieve, revise (proposed), approve and decline exceptions	Origination or full lifecycle of a mortgage application
Role-based operations, self-review prevention (proposed)	Rate calculation, affordability checks, loan underwriting
Immutable revisions, decision history, retry safety	Email notifications, workflow queues, enterprise IAM
Local seed identities and application IDs	Production-grade customer information and live banking integrations

2. Stakeholders, actors and business requirements

Actor	Responsibility
Relationship Manager	Creates exceptions; views requests; proposes an amended discount under a controlled revision workflow.
Reviewer	Approves/declines the exact current pending revision; cannot approve their own request (proposed).

Mortgage Application Process	Reads current approved discount for a referenced mortgage application.
Operations / engineering	Runs the service, investigates failures and validates traceability.

ID	Business requirement	Source
BR-01	Manager can request a discount for an existing mortgage application.	Required
BR-02	Authorized reviewer approves or declines and decision identifies actor/time.	Required
BR-03	Approved discount is available to the mortgage application process.	Required
BR-04	Current status and previous decisions can be inspected.	Required
BR-05	Duplicate requests from timeout/retry must be avoided.	Required
BR-06	Each approval is bound to immutable reviewed terms.	Proposed resolution of stakeholder conflict
BR-07	Service is runnable locally with instructions and realistic example data.	Required
BR-08	System provides role separation and concurrency protection.	Proposed engineering safeguard

3. User and functional requirements

ID	User requirement / system behavior	Acceptance rule
UR-01	Submit application ID, discount in basis points, and reason.	Successful create returns request ID, Pending and revision 1.
UR-02	Review the current pending revision.	Decision includes outcome, reviewer identity and UTC timestamp.
UR-03	Retrieve individual request and full history.	Current state, revision terms and prior decisions remain visible.
UR-04	Downstream reads only a current approved discount.	Declined/Pending/stale approvals never appear as usable discounts.
UR-05	Repeat a successful write after timeout.	Matching actor + scope + key + payload returns original result.
UR-06	Correct submitted terms without silently editing history.	New immutable revision, new version and re-review are required.
UR-07	Reject unauthorized or conflicting actions.	Appropriate 401/403/409 responses; no partial state changes.

3.1 Non-functional requirements

- Correctness: decisions apply only to the revision/version explicitly reviewed; concurrency races cannot produce two accepted outcomes.
- Auditability: record who did what and when; preserve the exact historical terms.
- Reliability: database transactions cover state changes, history and retry record atomically.
- Maintainability: straightforward layering, clean names, small service methods and automated tests.
- Security: derive actor identity and roles from authenticated context, not request JSON.
- Operability: local SQLite, seed data, documented start/test commands and structured error responses.
- Privacy: do not log sensitive customer reasons; expose generic server errors.

4. User scenarios and acceptance tests

Scenario	Given / When / Then
SC-01 Create	Given an eligible relationship manager and mortgage application ID; when they submit 50 bps and a reason; then request is Pending at revision 1 with Created history.
SC-02 Approve	Given a Pending request; when an authorized independent reviewer approves its current revision; then status is Approved, reviewer/time recorded and downstream reads 50 bps.
SC-03 Decline	Given a Pending request; when reviewer declines; then status is Declined, decision is logged and downstream has no current approval.
SC-04 Revise	Given any request owned by the manager; when manager submits changed terms using expected version/revision; then a new immutable revision becomes Pending and any previous approval is no longer current.
SC-05 Replay timeout	Given the first create succeeded but client timed out; when identical request with same Idempotency-Key is retried; then original response is replayed, with no duplicate.
SC-06 Key reused differently	Given an Idempotency-Key already bound to payload A; when it is reused for payload B; then return 409 without change.
SC-07 Competing actions	Given reviewers or revision and review act concurrently; when one update wins; then the other receives 409 rather than overwriting the newer state.
SC-08 Unauthorized/self review	Given a manager tries to decide their own request; then decision is forbidden and no history/decision is added.

4.1 Primary end-to-end workflow

```
Manager → POST create → Pending revision 1 → Reviewer GET → POST decision
        → Approved → MortgageProcess GET approved discount → apply in external system
        → Declined → no usable discount
Manager revise → revision 2 (Pending) → old approval retained in history, not in current lookup
```

5. Business decisions and ambiguities

5.1 Resolving the conflicting stakeholder statements

Decision (proposed): allow a relationship manager to change submitted terms ONLY by creating a new immutable revision. A reviewer approves the exact revision number and expected request version; approved terms can never be silently edited. A revision returns the request to Pending, making any previously approved rate unavailable to downstream consumers until the new revision is approved. This satisfies flexibility and preserves approval integrity.

Trade-off: revising an already approved request revokes the current effective approval. This is the supplied future entity design's default, not an explicit assignment policy. Confirm with stakeholders whether post-approval revisions are permitted, whether an approval can expire, and whether the mortgage process may already have consumed the old approval.

5.2 Other policies to confirm

- Discount range: propose 1–500 bps and required reason ≤2,000 characters, as in supplied entity design; not assignment-mandated.
- Cardinality: propose one exception request per mortgage application; confirm if multiple independent requests should be supported.

- Mortgage application existence: MVP uses known/seed application identifiers; a real integration must validate existence/eligibility.
- Read visibility: confirm which users may see another manager's requests and reasons.
- Approval consumption and expiry: confirm effective time, revocation and downstream snapshot rules.
- Retry retention: define production idempotency-key expiry and data protection.

6. Solution architecture

Deployment: one ASP.NET Core HTTP process, organized as three logical layers in one project, as recommended in the supplied future design. Layers are folder boundaries, not separate independently deployed services. A future larger system could split assemblies after the need is proven.

```
External actors / mortgage system
            ↓ HTTP + authentication
Presentation: Controllers + DTOs + ProblemDetails
            ↓ service interfaces
Business: application services + rules + domain models + repository contracts
            ↓ repository / unit-of-work contracts
DataAccess: EF Core + SQLite + transactions + idempotency + migrations
```

Layer	Main responsibilities	Example components
Presentation	Request validation, identity context, routing, HTTP semantics	PricingExceptionsController, DecisionDto, exception handler
Business	State transitions, authorization rules, revision/version checks, invariants	PricingExceptionService, PricingExceptionRequest, IRepository
DataAccess	Persistence, unique constraints, transactions, retry records	PricingDbContext, EF repositories, IdempotencyRecord

Dependency rule: controllers depend on business interfaces; business models must not depend on HTTP/EF types; DataAccess implements business repository abstractions. Program.cs composes dependencies.

7. Domain model and persistence design

Entity	Essential fields / constraints	Meaning
PricingExceptionRequest	Id, MortgageApplicationId, CreatorId, Status, Version, CreatedAtUtc, UpdatedAtUtc	Aggregate root and current workflow state.
RequestRevision	(RequestId, RevisionNumber) PK, DiscountBps, Reason, CreatedBy, CreatedAtUtc	Immutable business terms; sequence starts at 1.
ReviewDecision	Id, RequestId, RevisionNumber, Outcome, ReviewerId, DecidedAtUtc; unique (RequestId, RevisionNumber)	One decision per reviewed revision.
HistoryEntry	(RequestId, Sequence) PK, RevisionNumber, Action, ActorId, OccurredAtUtc	Append-only ordered audit trail.
User / UserRole	User ID, display name, distinct roles	Local seeded identities; production identity provider is separate.
IdempotencyRecord	ActorId, OperationScope, Key, PayloadHash, ResponseStatus, ResponseJson, Location	Technical record of completed successful writes.

Relationships: a request has many revisions and history entries; each revision has at most one decision. Creator/reviewer/actor IDs map to user identities. MortgageApplicationId is an external reference, with no required local MortgageApplication entity or foreign key.

Persistence choice (proposed): EF Core + SQLite gives persistent local data, relational uniqueness and real transactions. The source design describes EF Core InMemory as the existing implementation and SQLite as a future improvement; the Blue Harvest assignment does not mandate a database.

7.1 Domain invariants

- On create: revision 1, Pending status and Created history entry are committed together.
- Review allowed only for the current Pending revision, with matching expected version and revision number.
- One decision per revision; reviewer differs from request creator; reviewer role required.
- Revisions cannot overwrite historical revisions; a new revision resets current state to Pending.
- A current approved discount exists only when current revision is approved and request status is Approved.
- Each successful mutation increments the request Version; all timestamps are UTC.

8. REST API contract (proposed)

Method / route	Request / result	Permissions
POST /api/v1/pricing-exceptions	{ mortgageApplicationId, discountBps, reason } → 201 with ID/status	RelationshipManager
GET /api/v1/pricing-exceptions/{id}	Current terms, status, version, revision and decision	Authorized internal reader
POST /api/v1/pricing-exceptions/{id}/revisions	{ discountBps, reason, expectedVersion, expectedRevisionNumber }	Owning RelationshipManager
POST /api/v1/pricing-exceptions/{id}/decisions	{ outcome, expectedVersion, reviewedRevisionNumber }	Reviewer; not creator
GET /api/v1/pricing-exceptions/{id}/history	Ordered events, linked revisions and prior decisions	Authorized internal reader
GET /api/v1/mortgage-applications/{id}/approved-pricing-exception	Current approved discount or documented no-approval outcome	MortgageProcess

Require Idempotency-Key on POST writes. Obtain CreatorId/ReviewerId from authenticated claims, never request bodies. A local demo can use documented development-only identity simulation; production needs trusted authentication.

8.1 Example API exchange

```
POST /api/v1/pricing-exceptions
Idempotency-Key: create-app-1001-v1
{ "mortgageApplicationId": "<seed-guid>", "discountBps": 50,
  "reason": "Competitive rate requested" }
201 Created → { "id": "<request-guid>", "status": "Pending",
                "revisionNumber": 1, "version": 1 }
```

8.2 Error contract

Use RFC-style HTTP Problem Details: 400 malformed or invalid business inputs; 401 unauthenticated; 403 insufficient role or self-review; 404 unknown request; 409 duplicate application, stale revision/version, competing action or idempotency-key mismatch; 500 generic unexpected failure with a diagnostic trace ID. Choose and document the precise no-current-approval response (e.g., 404 Problem Details) and keep it consistent.

9. Implementation practices and controls

9.1 Transaction, concurrency and idempotency

- Wrap each write in one DB transaction: conditional aggregate update + revision/decision + history + completed idempotency response.
- Perform conditional update where Version matches expectedVersion; review also requires Pending and current revision match. Exactly one aggregate row must change.
- Enforce unique indexes as a second safeguard (request per application if confirmed; decision per revision; scoped idempotency key).
- Compute a canonical request payload hash, scoped to actor, operation and target. Exact retry replays the recorded successful response before evaluating current version.
- Reject same key with different payload as 409. If a transaction fails, no partial decision/history/retry entry remains.
- Use narrowly bounded retries for transient database contention; do not blindly retry business conflicts.

9.2 Secure coding and maintainable structure

- Use DTOs for input/output; keep domain entities out of public API contracts.
- Implement focused domain methods (Create, Revise, Decide) with explicit invariants; prefer readability over generic frameworks.
- Use dependency injection, async persistence operations, cancellation tokens and a time-provider abstraction for deterministic tests.
- Keep business services independent from ASP.NET Core and EF Core details.
- Use structured logs, correlation/trace IDs and sanitized fields; do not log full request reasons or user secrets.
- Document migration command, database path, seed identities, API explorer examples and reset instructions.

9.3 Suggested repository folders

```
src/PricingExceptions.Api/
  Presentation/{Controllers,Contracts,Middleware}/
  Business/{Models,Services,Abstractions,Exceptions}/
  DataAccess/{Persistence,Configurations,Repositories,Migrations}/
tests/PricingExceptions.Tests/{Unit,Integration}/
README.md  docs/{design.md,decisions.md,ai-development-log.md}
```

10. Test plan and quality gates

Test level	Examples	What it proves
Domain unit	Create/revise/decide, invalid bps/reason, immutable revisions	Business invariants and transitions.
Service unit	Owner/role rules, self-review, mismatched version/revision	Authorization policies and conflict paths.
SQLite integration	Unique indexes, transaction rollback, duplicate retry, persistence after restart	Real storage and atomicity behavior.
HTTP integration	201/200/400/401/403/404/409 and response DTOs	Public API correctness.
Concurrency integration	Two decisions race; decision versus revision race	Only one state transition succeeds.
Workflow test	Manager creates → reviewer approves → mortgage lookup → manager revises → no current approval	End-to-end behavior and downstream safety.

Quality gate: tests should run from a clean checkout without paid dependencies. A reviewer should be able to run the service locally, inspect OpenAPI, seed test actors, perform both approval and decline workflows, retry a request and review the ordered history.

11. Implementation plan and deliverables

Phase	Activities	Done when
1 — Foundation	Create Web API, folder boundaries, EF/SQLite configuration, migrations, seed data	Service starts locally; health/API endpoints reachable.
2 — Domain workflow	Implement aggregate, revisions, decisions, validation and authorization	Unit tests pass for main state transitions.
3 — Persistence safety	Implement repositories, transactions, optimistic concurrency, idempotency	SQLite integration/race tests pass.
4 — HTTP contract	Implement endpoints, DTO mapping, Problem Details, OpenAPI	Full scenario can be executed via HTTP.
5 — Submission	README, architectural decisions, test output, AI usage notes, time spent	Fresh clone instructions work; repository ready to share.

11.1 README essentials

- Problem summary, explicit assumptions and intentional non-goals.
- SDK/runtime prerequisites; dotnet restore, database setup, dotnet run and dotnet test commands.
- Demo user identities/roles, example mortgage application identifiers and example HTTP calls.
- Architecture overview, key trade-offs, idempotency behavior and state model.
- Testing scope, limitations and changes needed for production.
- Time spent and how AI tools or agents contributed (if used).

12. Agentic SDLC demonstration (bonus; controlled)

A useful bonus is an AI-assisted SDLC workflow, rather than embedding an unverified agent into a mortgage approval decision. For example, an analysis agent proposes test cases from user scenarios, a code-review agent detects missing concurrency/idempotency cases, and an engineer approves the changes. All generated tests run in CI and all generated code goes through human review.

Agent contribution	Verification / control	Residual risk
Draft requirements → test matrix	Human checks coverage against assignment and design decisions	Agent may invent unstated banking policies.
Generate boundary-case test suggestions	Run tests against real SQLite and inspect assertions	Plausible but weak tests may pass incorrectly.
Review code for state/race vulnerabilities	Static analysis, review and deliberate race tests	AI misses subtle transaction or auth defects.

Keep an AI work log: prompts/tasks, generated artifacts, manual changes, verification evidence and known limits. The assignment requires explanation of AI-assisted work, not unreviewed agent autonomy.

13. Pre-production hardening and open decisions

- Replace development identities with enterprise OIDC/JWT authentication and fine-grained authorization.
- Integrate existence/eligibility checks with the actual mortgage application system; define reliable downstream handoff, including whether an outbox/event is needed.
- Choose production database, migration/recovery strategy, backup, monitoring and idempotency retention policy.
- Define privacy classification, audit retention, access logs, security tests, rate limiting and secret management.

- Confirm all open business policies: rate cap, one request per application, post-approval changes, approval expiry/consumption and read visibility.
- Add deployment health probes, metrics, tracing, operational runbook and service-level objectives as appropriate.

14. Requirements traceability

Requirement	Design component	Validation
BR-01 / UR-01	Create endpoint + aggregate + revision 1	SC-01; HTTP/domain tests
BR-02 / UR-02	Decide endpoint + ReviewDecision + role guard	SC-02, SC-03, SC-08
BR-03 / UR-04	Approved-discount read endpoint	SC-02–04, workflow test
BR-04 / UR-03	HistoryEntry + immutable revisions + history endpoint	SC-01–04; ordered-history integration test
BR-05 / UR-05	IdempotencyRecord + unique key + transaction	SC-05, SC-06
BR-06 / UR-06	Revision number + optimistic Version + reset to Pending	SC-04, SC-07
BR-07	SQLite seeds + README + demo scripts	Fresh-clone smoke test

15. Source references and status

Source A: Blue Harvest Agentic SDLC Engineering Assignment, pp. 1–3. Requirements, deliverables, constraints and stakeholder conflict are drawn from this assignment.

Source B: Pricing Exceptions — Entity and Application Design, prepared 2 Nov 2026, pp. 1–5. Proposed architecture, entities, revision rules, endpoints, SQLite choice and technical controls build upon this earlier future design.

The earlier sections record the original proposal. The current-code appendix below supersedes those sections for implemented architecture and method names; SQLite and repository/unit-of-work abstractions in the proposal are not the current implementation.

Current implementation and method reference (7 November 2026)

This appendix reflects the implemented ASP.NET Core API, React UI and PostgreSQL deployment. The changes documented here rename methods and variables; they do not change HTTP routes, DTOs, authorization, database schema, idempotency scopes or business rules.

Implemented architecture and workflow

Controllers call service interfaces directly. PricingExceptionService owns creation, revisions, decisions and history; UserService resolves stored identities and enforces roles; ApprovedDiscountService reads the currently approved discount. Services use PricingExceptionsDbContext directly. There are no separate repository or unit-of-work services. PostgreSQL is the normal provider; isolated tests explicitly select InMemory.

Create submits revision 1 as Pending. The assigned independent reviewer approves or declines the current revision. The creator may submit new immutable terms, returning the request to Pending. Old terms and decisions remain in history. Only an approval on the current revision is effective.

Current public methods and private helpers

PricingExceptionsController

- CreatePricingExceptionRequestAsync (public)
- GetPricingExceptionRequestsAsync (public)
- GetPricingExceptionRequestByIdAsync (public)
- RecordPricingExceptionDecisionAsync (public)
- SubmitPricingExceptionRevisionAsync (public)
- GetPricingExceptionHistoryAsync (public)

UsersController

- GetCurrentUserProfileAsync (public)
- GetReviewersAsync (public)

ApprovedDiscountsController

- GetApprovedDiscountForMortgageApplicationAsync (public)

PricingExceptionService

- CreatePricingExceptionRequestAsync (public)
- SubmitRequestRevisionAsync (public)
- ReviewPricingExceptionRequestAsync (public)
- ListPricingExceptionRequestsAsync (public)
- GetPricingExceptionRequestAsync (public)
- GetRequestHistoryAsync (public)
- GetRequestRevisionsAsync (public)
- GetReviewDecisionsAsync (public)
- GetRequestHistorySnapshotAsync (public)
- ExecuteAuthorizedRequestReadAsync (private)
- FindRequestOrThrowAsync (private)
- ExecuteIdempotentWriteAsync (private)
- ValidateDiscountTerms (private)

- ValidateRequestVersionAndRevision (private)
- CreateRequestRevision (private)
- AppendRequestHistoryEntry (private)
- MapToPricingExceptionResult (private)

UserService

- GetReviewersAsync (public)
- GetUserOrThrowAsync (public)
- GetUserWithRequiredRoleAsync (public)

ApprovedDiscountService

- GetAsync (public)

Entity model

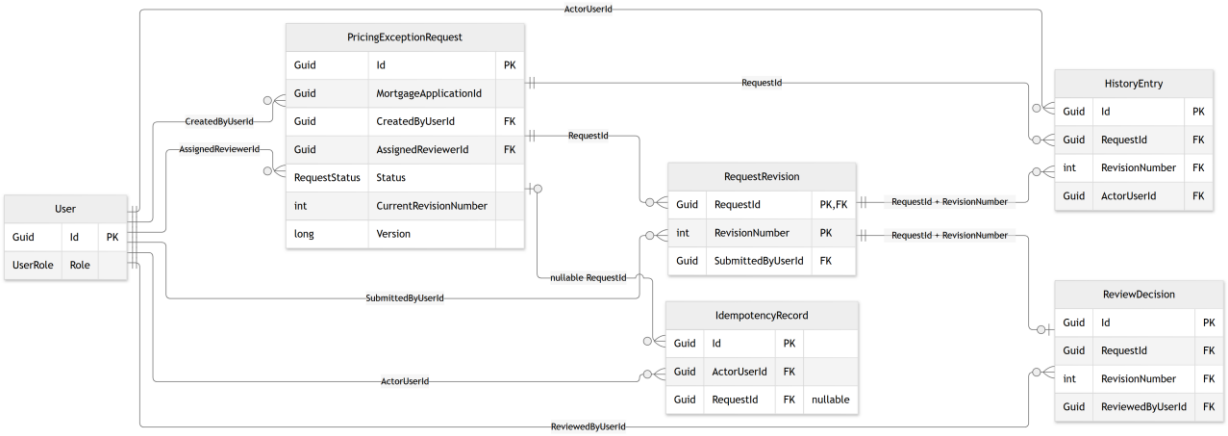
ApprovedDiscountService.GetAsync is intentionally unchanged. PricingExceptionsController uses ActionName on GetPricingExceptionRequestByIdAsync so CreatedAtAction continues to produce the request Location header.

Validation evidence

Latest local validation: backend build succeeded with zero warnings and errors; 184 backend tests passed (56 unit, 118 HTTP integration, 10 PostgreSQL); frontend build/lint and 20 browser tests passed. The Docker HTTP workflow, persistence across recreation and one deployed multi-user browser workflow also passed. These checks do not prove production readiness.

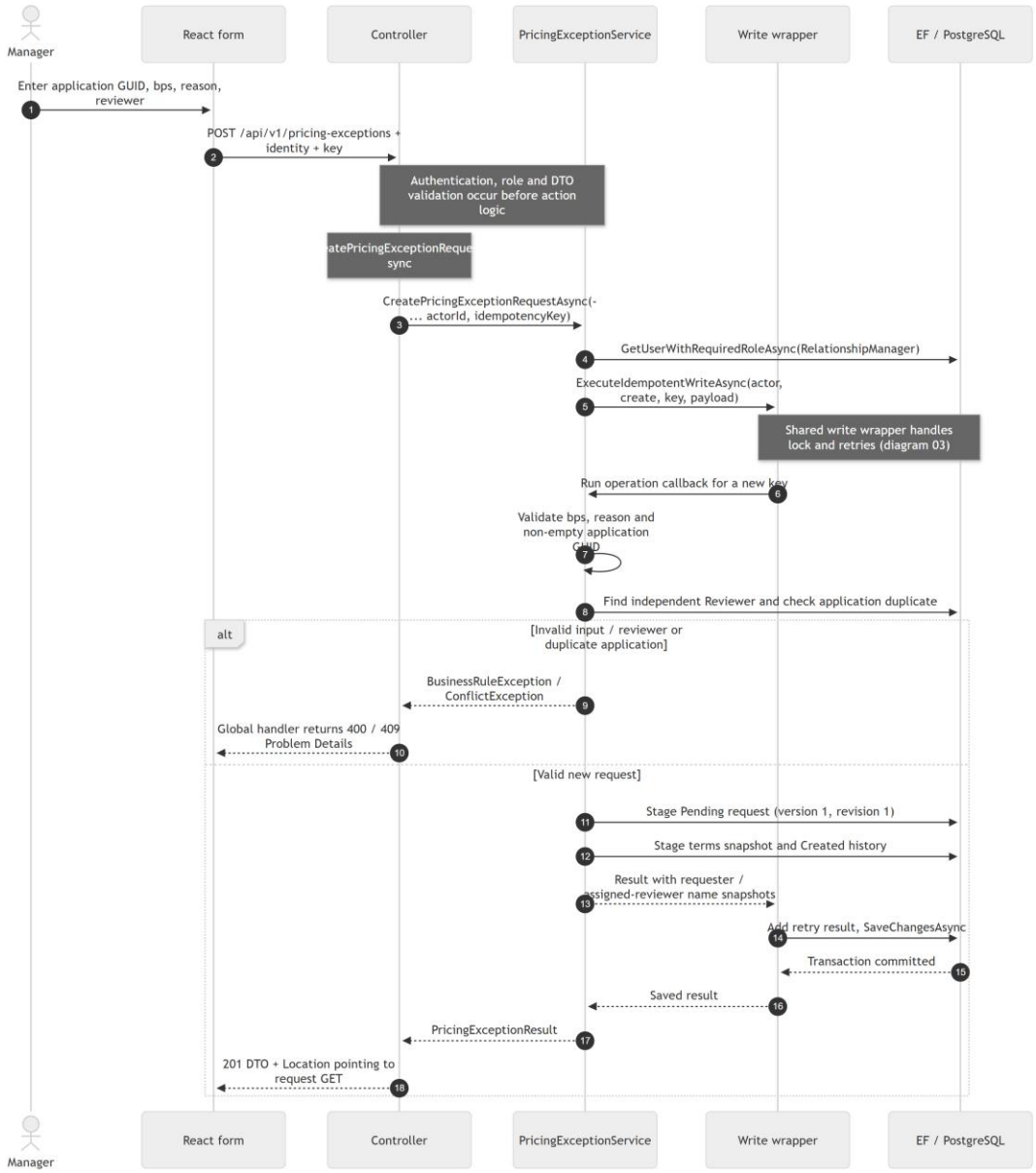
Current entity relationships

Six persisted entities; unchanged by the naming refactor. MortgageApplicationId remains an external reference. Methods are documented in services and sequences rather than added as entity fields.



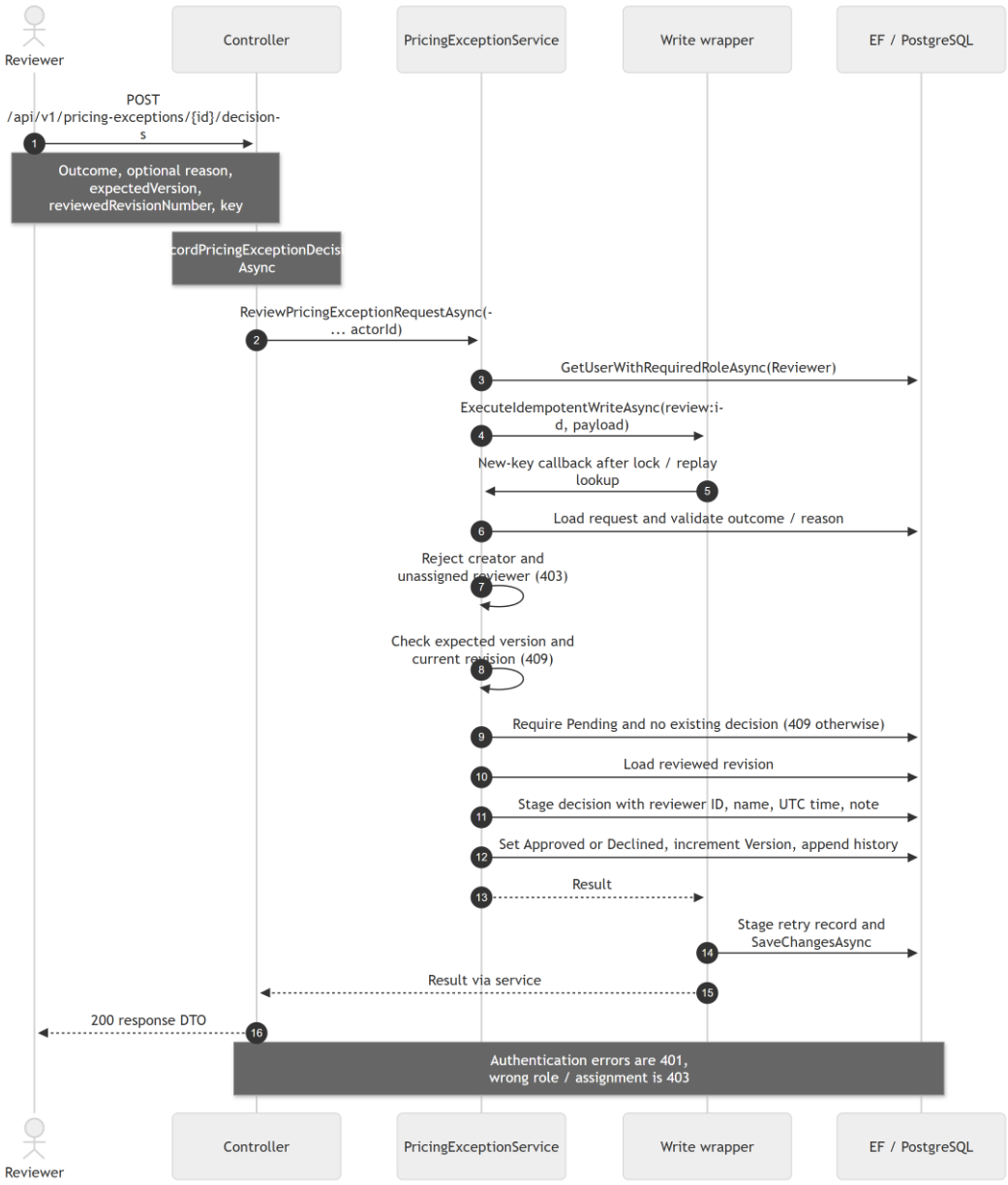
The standalone entity document contains the full property dictionary and constraint caveats. See CurrentImplementation/EntityRelationships.md and the full-field SVG for editable detail.

Create and submit — current method names



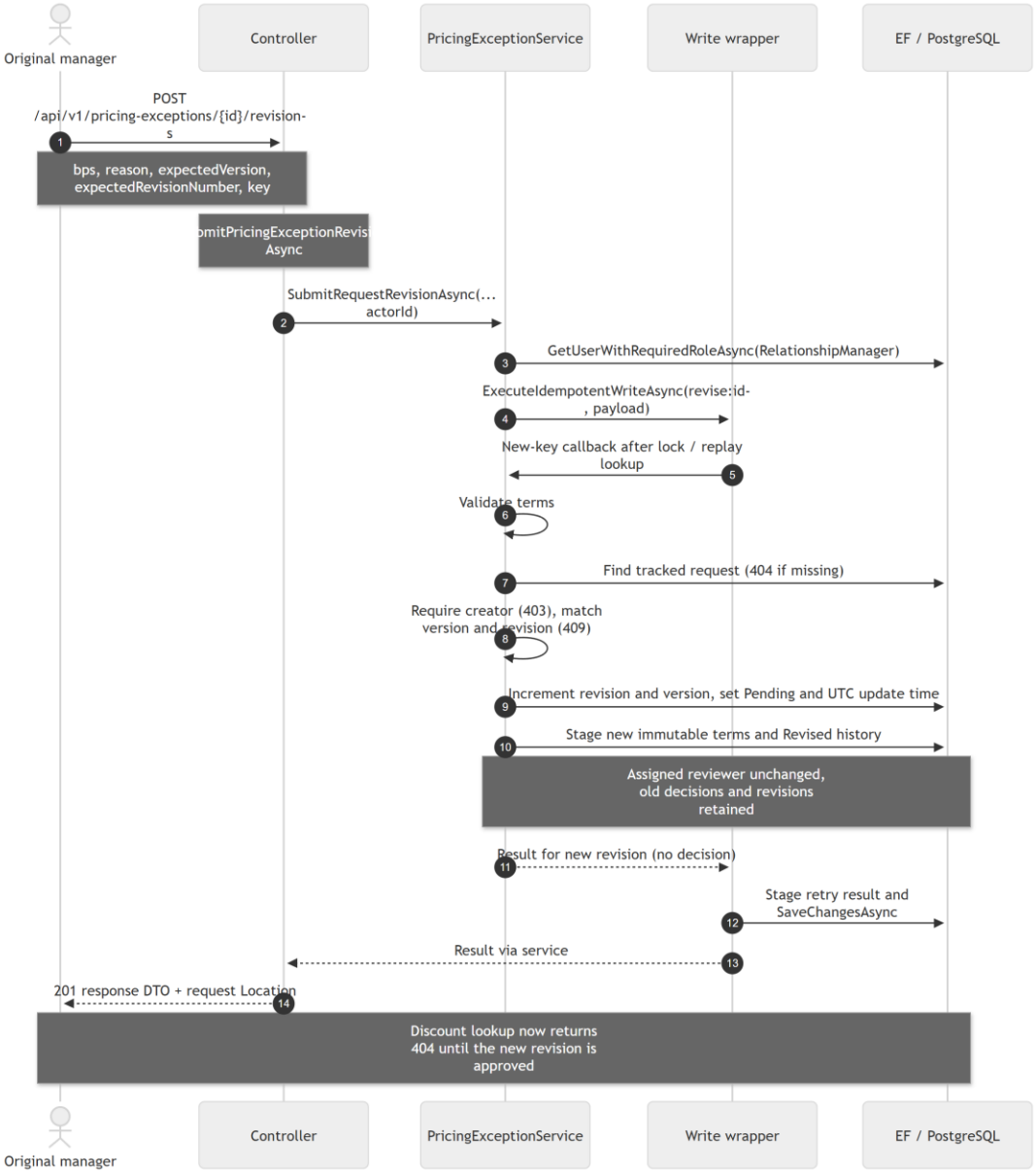
All eight workflow diagrams, source references and implementation limits are maintained in BlueHarvest_Current_Sequence_Diagrams_Final.docx and its PDF companion.

Record a decision — current method names



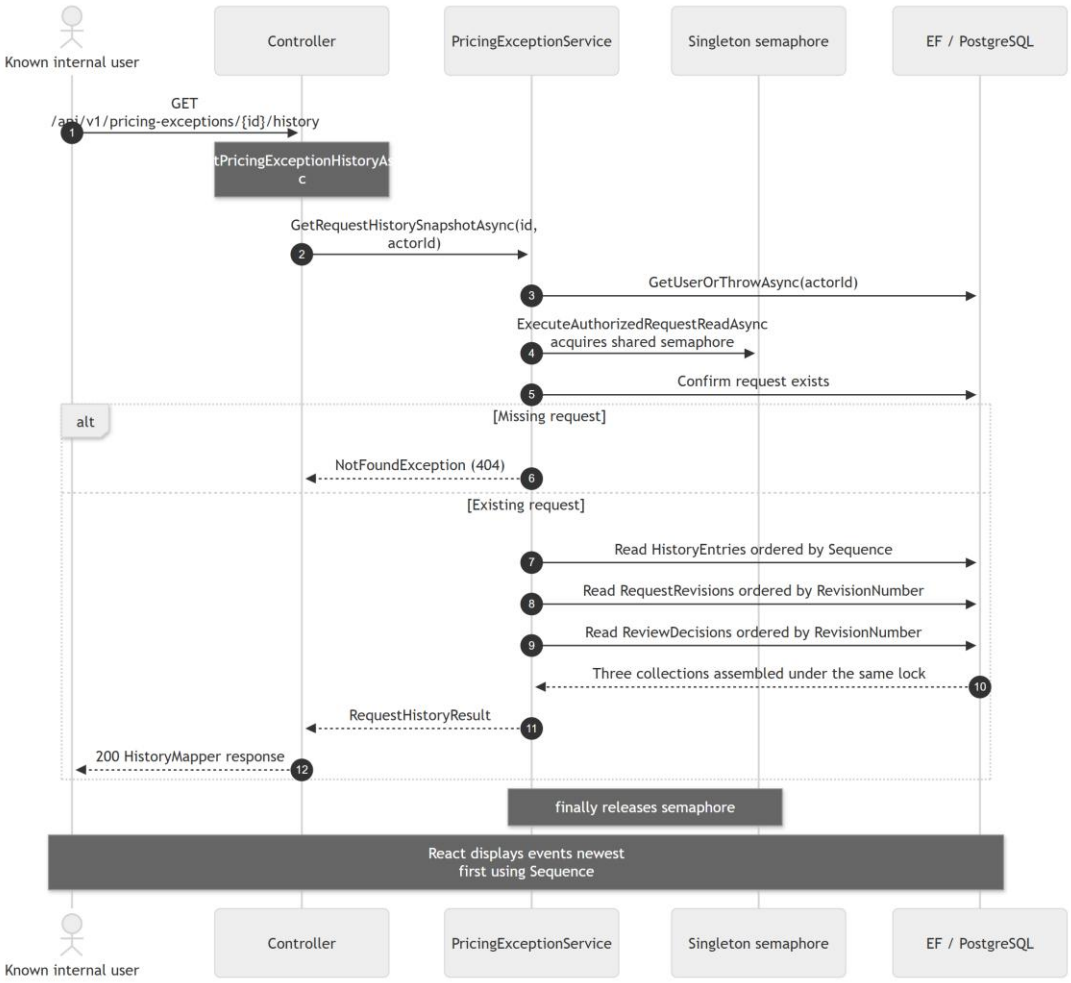
All eight workflow diagrams, source references and implementation limits are maintained in BlueHarvest_Current_Sequence_Diagrams_Final.docx and its PDF companion.

Submit a revision — current method names



All eight workflow diagrams, source references and implementation limits are maintained in BlueHarvest_Current_Sequence_Diagrams_Final.docx and its PDF companion.

Read request history — current method names



All eight workflow diagrams, source references and implementation limits are maintained in BlueHarvest_Current_Sequence_Diagrams_Final.docx and its PDF companion.